

Mnemos

Technical Documentation

An Agentic Memory System with Provenance-Tracked Consolidation

Generated Codebase Documentation

July 16, 2026

Abstract

Mnemos is a self-hosted, single-user *agentic memory system* that gives large-language-model assistants persistent, self-consolidating, provenance-tracked memory. It is implemented as a FastAPI backend with a LangGraph-orchestrated “sleep cycle” for memory consolidation, an embedded Qdrant vector store paired with a SQLite relational store for full audit trails, a multi-provider LLM abstraction (Groq, Gemini, Ollama), a FastMCP server exposing memory operations as tools to Claude Desktop / Cursor, and a React single-page frontend for chat, memory inspection, roadmap planning, and consolidation-log review. This document provides an exhaustive technical reference to every subsystem: data models, algorithms (decay-ranked retrieval, contradiction resolution, exponential moving-average confidence updates), API contracts, the two LangGraph state machines, the frontend component tree, and the deployment topology.

Contents

1	Overview	3
1.1	What Mnemos Is	3
1.2	High-Level Architecture	3
1.3	Repository Layout	3
2	Data Model	5
2.1	Episode	5
2.2	Fact	5
2.3	GoalFact	6
2.4	Provenance and Log Models	6
2.5	Entity-Relationship Diagram	6
3	Storage Layer	7
3.1	SQLite Store (src/store/sqlite_store.py, 272 lines)	7
3.1.1	Schema	7
3.1.2	CRUD Surface	8
3.2	Vector Store (src/store/vector_store.py, 76 lines)	9
3.3	Embeddings (src/store/embeddings.py, 33 lines)	9
4	Memory Subsystem	11
4.1	Decay-Ranked Retrieval (src/memory/retrieval.py, 59 lines)	11
4.1.1	Formula	11
4.1.2	Implementation	11
4.2	Contradiction Detection & Resolution (src/memory/contradiction.py, 108 lines)	12
4.2.1	Two-Stage Detection	12
4.2.2	Resolution Policy	12
4.3	Consolidation — the “Sleep Cycle” (src/memory/consolidation.py, 265 lines)	13
4.3.1	State Schema	13
4.3.2	Graph Wiring	13
4.3.3	Node 1 — <code>_extract</code>	14
4.3.4	Node 2 — <code>_dedupe</code>	14
4.3.5	Node 3 — <code>_contradiction_check</code>	14
4.3.6	Node 4 — <code>_prune</code>	15
4.3.7	Entry Point	15
4.4	Why Provenance Never Breaks	15
5	LLM Client Abstraction	16
5.1	Uniform Interface (src/llm_client.py, 160 lines)	16
5.2	Provider Implementations	16
6	HTTP API (src/api.py)	17
6.1	Request/Response Models	17
6.2	Endpoint Reference	17

6.3	Auto-Consolidation Trigger	18
6.4	Ingestion Duplication	19
7	MCP Server	20
7.1	Purpose and Transport	20
7.2	Tools	20
7.3	Resources	21
8	Roadmap Planner	22
8.1	Purpose	22
8.2	State Schema	22
8.3	Graph Wiring	22
8.4	Node 1 — Researcher	22
8.5	Node 2 — Synthesizer	23
8.6	Post-Processing (plan_roadmap)	23
8.7	Research Tools (src/planner/tools.py, 182 lines)	23
9	Configuration	25
9.1	Loader (src/config.py, 41 lines)	25
9.2	Full Tunable Reference	25
10	Frontend	27
10.1	Application Shell (App.jsx, 59 lines)	27
10.2	API Layer (api.js, 65 lines)	27
10.3	Panels	28
10.3.1	Chat (panels/Chat.jsx, 301 lines)	28
10.3.2	Memory Inspector (panels/MemoryInspector.jsx, 188 lines)	28
10.3.3	Goals (panels/Goals.jsx, 122 lines)	28
10.3.4	Consolidation Log (panels/ConsolidationLog.jsx, 135 lines)	28
10.4	Design System (index.css, 68 lines)	28
11	Deployment & Tooling	30
11.1	Docker Compose Topology	30
11.2	Backend Dockerfile	30
11.3	Frontend Dockerfile	30
11.4	Local Development (Makefile)	31
11.5	Python Dependencies (pyproject.toml)	31
11.6	Environment Variables	31
12	Cross-Cutting Observations	32

Chapter 1

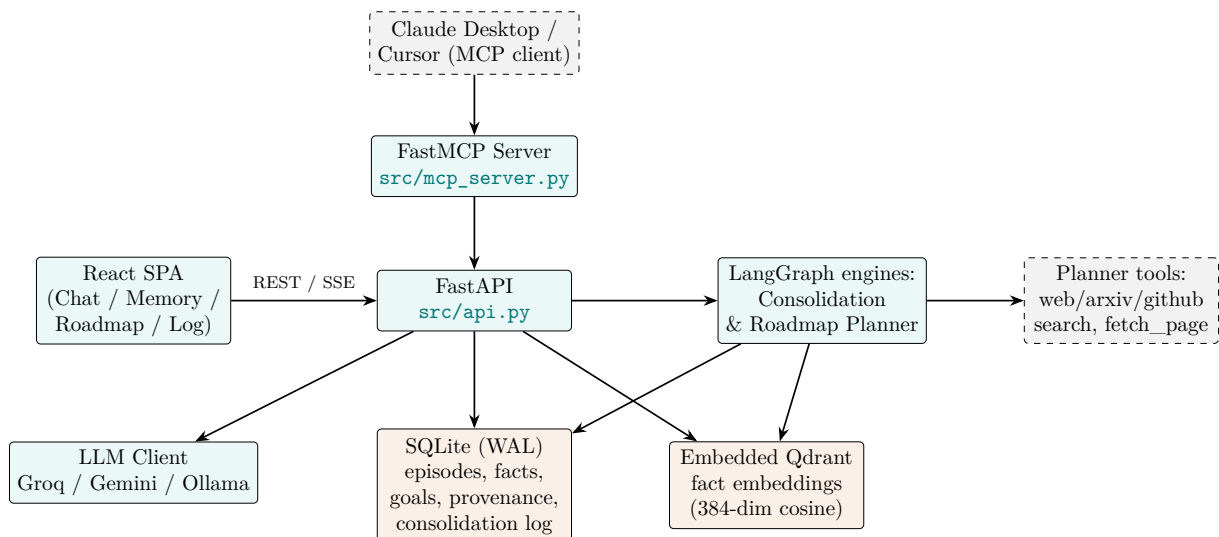
Overview

1.1 What Mnemos Is

Mnemos gives an AI assistant a memory that behaves less like a chat log and more like a human's: raw experiences (*episodes*) are periodically distilled into durable, confidence-weighted *facts*; facts that conflict are adjudicated and superseded rather than silently overwritten; facts that go unused decay in relevance over time; and every fact retains a full chain of provenance back to the episode(s) that produced it.

The system is intentionally **single-tenant** — there is one implicit user (`config: user.default_user_id`, `default local_user`) throughout the codebase, with no authentication layer. This is a deliberate simplification appropriate for a personal-assistant deployment (Docker Compose on a single host, or an MCP server running locally beside Claude Desktop).

1.2 High-Level Architecture



1.3 Repository Layout

Path	Purpose
src/api.py	All FastAPI endpoints (single file, no routers)
src/schema.py	Pydantic v2 domain models

Path	Purpose
<code>src/config.py, config.yaml</code>	Configuration loader and tunables
<code>src/llm_client.py</code>	Multi-provider LLM abstraction
<code>src/mcp_server.py</code>	FastMCP server (8 tools, 2 resources)
<code>src/memory/consolidation.py</code>	LangGraph “sleep cycle”
<code>src/memory/contradiction.py</code>	Contradiction detection & resolution
<code>src/memory/retrieval.py</code>	Decay-ranked vector retrieval
<code>src/planner/roadmap_planner.py</code>	Two-node LangGraph roadmap planner
<code>src/planner/tools.py</code>	Research tools bound to the planner LLM
<code>src/store/sqlite_store.py</code>	Full CRUD over SQLite (WAL mode)
<code>src/store/vector_store.py</code>	Embedded Qdrant wrapper
<code>src/store/embeddings.py</code>	SentenceTransformer embedding & cosine similarity
<code>frontend/src/App.jsx</code>	Tab shell: Chat / Memory / Roadmap / Sleep
<code>frontend/src/api.js</code>	All frontend API calls (axios + SSE fetch)
<code>frontend/src/panels/*.jsx</code>	Chat, MemoryInspector, Goals, ConsolidationLog
<code>frontend/src/index.css</code>	Design tokens (HSL variables, zero-radius terminal aesthetic)
<code>docker-compose.yml, Dockerfiles</code>	Container topology
<code>Makefile</code>	Local dev orchestration
<code>pyproject.toml</code>	Python dependencies (uv/hatchling)

Chapter 2

Data Model

All domain models are Pydantic v2 `BaseModel` subclasses defined in [src/schema.py](#) (68 lines). Identifiers are all `uuid4` strings generated by a shared `new_id()` helper. Two constrained literal types anchor the type system:

```
1 FactType = Literal["preference", "status", "event", "skill", "goal", "other"]
2 GoalStatus = Literal["not_started", "in_progress", "done"]
```

Listing 2.1: Core literal types (schema.py)

2.1 Episode

Raw episodic memory — one turn of interaction or one ingested document.

Field	Type	Default	Notes
<code>episode_id</code>	<code>str</code>	<code>uuid4()</code>	Primary key
<code>user_id</code>	<code>str</code>	—	Always the single configured user
<code>session_id</code>	<code>str</code>	—	Groups turns of one conversation
<code>text</code>	<code>str</code>	—	Free text; chat turns stored as "User: ... \nAssistant: ..."
<code>timestamp</code>	<code>datetime</code>	<code>utcnow()</code>	

2.2 Fact

A consolidated, confidence-weighted unit of long-term memory.

Field	Type	Default	Notes
<code>fact_id</code>	<code>str</code>	<code>uuid4()</code>	Primary key
<code>content</code>	<code>str</code>	—	Full declarative sentence
<code>type</code>	<code>FactType</code>	<code>"other"</code>	One of the 6 enum values
<code>confidence</code>	<code>float</code>	<code>1.0</code>	$\in [0, 1]$; updated via EMA (§4.3.4)
<code>last_seen</code>	<code>datetime</code>	<code>utcnow()</code>	Refreshed on every re-observation
<code>metadata</code>	<code>dict</code>	<code>{}</code>	Free-form
<code>superseded_by</code>	<code>Optional[str]</code>	<code>None</code>	Fact ID of the winner, or literal "pruned"
<code>flagged</code>	<code>bool</code>	<code>False</code>	Set <code>True</code> on an unresolved true-tie contradiction

Facts are never hard-deleted. Both pruning (confidence below floor) and contradiction resolution mark `superseded_by` rather than removing the row — this is the load-bearing design decision that keeps provenance chains intact indefinitely (§4.4).

2.3 GoalFact

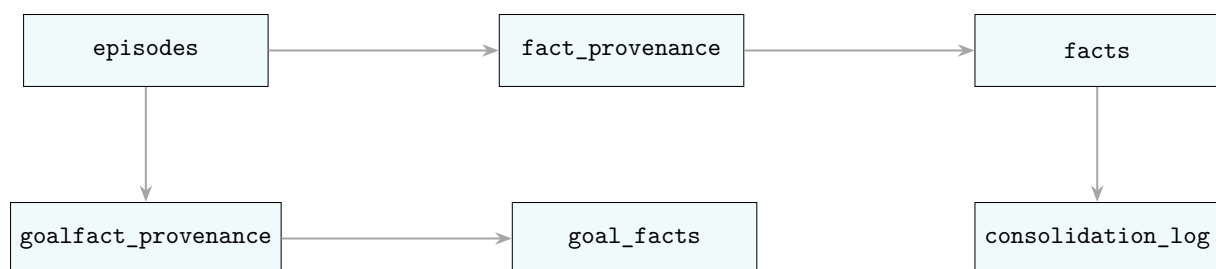
One phase of a generated learning roadmap.

Field	Type	Default	Notes
<code>goal_id</code>	<code>str</code>	<code>uuid4()</code>	Primary key
<code>user_id</code>	<code>str</code>	—	
<code>topic</code>	<code>str</code>	—	Roadmap topic, groups phases
<code>phase_index</code>	<code>int</code>	—	Ordering within a topic
<code>phase_content</code>	<code>str</code>	—	Description of the phase
<code>status</code>	<code>GoalStatus</code>	<code>"not_started"</code>	Kanban-style status
<code>metadata</code>	<code>dict</code>	—	Holds <code>resources: list[str]</code> , <code>prerequisites: str</code>

2.4 Provenance and Log Models

- **FactProvenance** — junction row (`fact_id`, `episode_id`).
- **GoalFactProvenance** — junction row (`goal_id`, `episode_id`).
- **ConsolidationLogEntry** — audit record of one sleep-cycle run: `run_id`, `timestamp`, `user_id`, `episodes_processed`, `facts_created`, `facts_updated`, `contradictions_resolved`, `facts_pruned`, and `details: list[dict]` — a free-form event trail (e.g. `{"type": "contradiction_resolved", ...}` or `{"type": "pruned", ...}`).

2.5 Entity-Relationship Diagram



Chapter 3

Storage Layer

Mnemos uses two independent, purpose-fit stores rather than one general database: SQLite for relational/audit data, and an embedded Qdrant instance purely for vector similarity search. There is no ORM — `sqlite_store.py` issues raw SQL via the standard-library `sqlite3` module.

3.1 SQLite Store (`src/store/sqlite_store.py`, 272 lines)

Every call opens a connection through a context manager:

```
1 @contextmanager
2 def _conn():
3     conn = sqlite3.connect(_DB_PATH)
4     conn.row_factory = sqlite3.Row
5     conn.execute("PRAGMA journal_mode=WAL")
6     conn.execute("PRAGMA foreign_keys=ON")
7     try:
8         yield conn
9         conn.commit()
10    except Exception:
11        conn.rollback()
12        raise
13    finally:
14        conn.close()
```

Listing 3.1: Connection handling

WAL (write-ahead logging) mode allows concurrent readers alongside a writer, which matters because the FastAPI process and any MCP server process may touch the same database file simultaneously.

3.1.1 Schema

```
1 CREATE TABLE IF NOT EXISTS episodes (
2     episode_id TEXT PRIMARY KEY,
3     user_id     TEXT,
4     session_id TEXT,
5     text        TEXT,
6     timestamp   TEXT
7 );
8
9 CREATE TABLE IF NOT EXISTS facts (
10    fact_id      TEXT PRIMARY KEY,
11    content      TEXT,
12    type         TEXT DEFAULT 'other',
13    confidence    REAL DEFAULT 1.0,
```

```

14     last_seen      TEXT,
15     metadata       TEXT DEFAULT '{}',
16     superseded_by  TEXT,
17     flagged        INTEGER DEFAULT 0
18 );
19
20 CREATE TABLE IF NOT EXISTS goal_facts (
21     goal_id        TEXT PRIMARY KEY,
22     user_id        TEXT,
23     topic          TEXT,
24     phase_index    INTEGER,
25     phase_content  TEXT,
26     status         TEXT DEFAULT 'not_started',
27     metadata       TEXT DEFAULT '{}',
28 );
29
30 CREATE TABLE IF NOT EXISTS fact_provenance (
31     fact_id        TEXT,
32     episode_id     TEXT,
33     PRIMARY KEY (fact_id, episode_id)
34 );
35
36 CREATE TABLE IF NOT EXISTS goalfact_provenance (
37     goal_id        TEXT,
38     episode_id     TEXT,
39     PRIMARY KEY (goal_id, episode_id)
40 );
41
42 CREATE TABLE IF NOT EXISTS consolidation_log (
43     run_id         TEXT PRIMARY KEY,
44     timestamp      TEXT,
45     user_id        TEXT,
46     episodes_processed INTEGER,
47     facts_created  INTEGER,
48     facts_updated  INTEGER,
49     contradictions_resolved INTEGER,
50     facts_pruned   INTEGER,
51     details        TEXT DEFAULT '[]',
52 );
53
54 CREATE INDEX IF NOT EXISTS idx_episodes_user ON episodes(user_id);
55 CREATE INDEX IF NOT EXISTS idx_facts_user_conf ON facts(confidence);
56 CREATE INDEX IF NOT EXISTS idx_goal_user ON goal_facts(user_id);

```

Listing 3.2: Table definitions (init_db)

Note: `idx_facts_user_conf` is named as though it indexes `user_id`, but it actually indexes `confidence` — a naming/implementation mismatch worth fixing if `facts` ever needs per-user filtering at scale (currently moot under the single-tenant model).

3.1.2 CRUD Surface

All writes use `INSERT OR REPLACE`, giving upsert semantics keyed on the primary key. Row→model conversion happens through private `_row_to_*` helpers that JSON-decode the `metadata/details` columns and ISO-parse timestamps.

Function	Behavior
<code>save_episode(episode)</code>	Upsert one episode row
<code>get_episodes(user_id, limit)</code>	Most recent N episodes

Function	Behavior
<code>get_episodes_since(user_id, since)</code>	Episodes newer than a timestamp (drives incremental consolidation)
<code>save_fact(fact)</code>	Upsert one fact row
<code>get_facts(include_superseded)</code>	All facts, optionally filtering out superseded/pruned ones
<code>get_fact(fact_id)</code>	Single fact lookup
<code>save_goal_fact(goal) / get_goal_facts(user_id)</code>	Ordered by (topic, phase_index)
<code>save_fact_provenance</code>	INSERT OR IGNORE — idempotent link
<code>get_fact_provenance(fact_id)</code>	Returns <code>list[str]</code> of episode IDs
<code>save_goalfact_provenance</code>	Same idempotent pattern for goals
<code>save_consolidation_log / get_consolidation_log(user_id, limit=20)</code>	Audit history

3.2 Vector Store ([src/store/vector_store.py](#), 76 lines)

Wraps an **embedded** Qdrant client (`QdrantClient(path=...)`, on-disk, no server process) lazily created and memoized in a module global. The collection is created on first use with `VectorParams(size=384, distance=Distance.COSINE)`.

Qdrant point IDs must be integers or UUIDs; since fact IDs are UUID *strings*, they are deterministically folded into a 63-bit unsigned integer:

```
1 def _id_to_uint(uuid_str: str) -> int:
2     return uuid.UUID(uuid_str).int % (2 ** 63)
```

Listing 3.3: UUID-to-point-ID conversion

Function	Behavior
<code>upsert_fact(fact_id, embedding, payload)</code>	Stores the vector with payload merged with <code>{"fact_id": fact_id}</code>
<code>search_similar(query_embedding, top_k=10, score_threshold=0.0)</code>	Returns <code>[{fact_id, score, payload}, ...]</code> via <code>client.query_points()</code>
<code>delete_fact(fact_id)</code>	Removes the point by its converted ID

3.3 Embeddings ([src/store/embeddings.py](#), 33 lines)

Model: **sentence-transformers/all-MiniLM-L6-v2** — 384-dimensional, CPU-only, lazily instantiated once via `@lru_cache(maxsize=1)`.

```
1 def embed(text: str) -> list[float]:
2     return _model().encode(text, normalize_embeddings=True).tolist()
3
4 def embed_batch(texts: list[str]) -> list[list[float]]:
5     return _model().encode(texts, batch_size=32,
6                             normalize_embeddings=True).tolist()
7
8 def cosine_similarity(a, b) -> float:
9     return float(np.dot(a, b)) # valid because both vectors are L2-normalized
```

Listing 3.4: Embedding and similarity

Because embeddings are pre-normalized, cosine similarity degenerates to a plain dot product — this is why `cosine_similarity` appears throughout the codebase as a single `np.dot` call.

Chapter 4

Memory Subsystem

This is the conceptual core of Mnemos: turning a stream of episodes into a durable, decaying, contradiction-resolved fact base. Three modules cooperate — retrieval, contradiction, and consolidation.

4.1 Decay-Ranked Retrieval ([src/memory/retrieval.py](#), 59 lines)

Retrieval combines vector similarity with a recency-decay penalty so that older, unused facts naturally fade from context even if they remain semantically relevant.

4.1.1 Formula

Let q be the query embedding, f a candidate fact with embedding v_f and `last_seen` timestamp t_f . Let Δt_f be the age of the fact in days at query time. The half-life H (default 30 days, `memory.decay_half_life_days`) defines a decay constant:

$$\lambda = \frac{\ln 2}{H}$$

and the final ranking score is

$$\text{score}(f) = \underbrace{\cos(q, v_f)}_{\text{semantic similarity}} \times \underbrace{e^{-\lambda \Delta t_f}}_{\text{exponential recency decay}}$$

This halves a fact’s effective relevance every H days regardless of how similar it remains to new queries — modeling forgetting as compounding decay rather than a hard cutoff.

4.1.2 Implementation

```
1 _HALF_LIFE_DAYS = get("memory.decay_half_life_days", 30)
2 _LAMBDA = math.log(2) / _HALF_LIFE_DAYS
3
4 def retrieve(query, top_k=5, include_superseded=False):
5     query_vec = embeddings.embed(query)
6     candidates = vector_store.search_similar(query_vec, top_k=top_k * 3)
7     results = []
8     for c in candidates:
9         fact = sqlite_store.get_fact(c["fact_id"])
10        if fact is None or (not include_superseded and fact.superseded_by):
11            continue
12        age = _age_days(fact) # now - last_seen, in days
13        score = c["score"] * math.exp(-_LAMBDA * age)
14        results.append((fact, score))
```

```

15     results.sort(key=lambda x: x[1], reverse=True)
16     return results[:top_k]

```

Listing 4.1: retrieve() — over-fetch then decay-rerank

Two implementation details matter:

1. Qdrant is asked for $3\times$ the requested `top_k` candidates, because Qdrant itself only ranks by raw cosine similarity — the decay re-ranking happens afterward in Python, and over-fetching guards against a fact that is highly similar but old being pushed out of the candidate pool before decay can be applied.
2. Any exception from the vector store (e.g. the collection not existing yet, before the first consolidation run) is caught and yields an empty result rather than propagating — chat still functions with no memory context on a cold start.

`retrieve_for_context(query, top_k)` is a thin wrapper returning just `list[Fact]` (dropping scores) for direct injection into the chat system prompt.

4.2 Contradiction Detection & Resolution (`src/memory/contradiction.py`, 108 lines)

4.2.1 Two-Stage Detection

A cheap embedding-similarity filter gates an expensive LLM judgment call, so the LLM is only invoked for genuinely plausible conflicts:

```

1  def find_contradictions(new_fact: Fact, existing_facts: list[Fact]) -> list[
    Fact]:
2      new_vec = embeddings.embed(new_fact.content)
3      conflicts = []
4      for ef in existing_facts:
5          if ef.fact_id == new_fact.fact_id:
6              continue
7          ef_vec = embeddings.embed(ef.content)
8          sim = embeddings.cosine_similarity(new_vec, ef_vec)
9          if sim >= _SIM_THRESHOLD:                # config: memory.
              similarity_threshold (0.60)
10             if _judge_contradiction(new_fact.content, ef.content):
11                 conflicts.append(ef)
12     return conflicts

```

Listing 4.2: find_contradictions()

The similarity threshold is configured at `memory.similarity_threshold` (0.60 in `config.yaml`); the module also defines a code-level fallback default of 0.85 that is only used if the config key is absent. The LLM judge (`_judge_contradiction`) is few-shot-prompted to distinguish genuine mutual exclusivity (e.g. “*is a junior engineer*” vs. “*is a senior engineer*”) from compatible statements that merely sound related (e.g. “*is vegan*” vs. “*doesn’t eat meat*”), and must reply with a bare CONTRADICT or CONSISTENT token.

4.2.2 Resolution Policy

Given a confirmed contradiction between a new fact and an existing one, resolution proceeds by a strict precedence: confidence, then recency, then an explicit tie state.

```

1  def resolve(new_fact, existing):
2      if new_fact.confidence > existing.confidence:
3          winner, loser = new_fact, existing

```

```

4     elif existing.confidence > new_fact.confidence:
5         winner, loser = existing, new_fact
6     else:
7         if new_fact.last_seen > existing.last_seen:
8             winner, loser = new_fact, existing
9         elif existing.last_seen > new_fact.last_seen:
10            winner, loser = existing, new_fact
11        else:
12            # true tie: flag both, keep both, no supersession
13            new_fact.flagged = True
14            existing.flagged = True
15            save(new_fact); save(existing)
16            return new_fact, existing
17
18    loser.superseded_by = winner.fact_id    # soft supersession, never
19            deleted
20    save(winner); save(loser)
21    return winner, loser

```

Listing 4.3: resolve() precedence logic

The loser is never hard-deleted — its row persists with `superseded_by` set to the winner’s ID, so provenance queries can still trace back through it. A genuine tie (equal confidence *and* equal `last_seen`) sets `flagged = True` on both facts and keeps them side by side for a human to resolve later, surfaced in the UI’s fact list.

4.3 Consolidation — the “Sleep Cycle” ([src/memory/consolidation.py](#), 265 lines)

Consolidation is a four-node linear LangGraph that turns a batch of episodes into vetted facts.

4.3.1 State Schema

```

1 class ConsolidationState(TypedDict):
2     user_id: str
3     episodes: list[Episode]
4     extracted: list[dict]
5     new_facts: list[Fact]
6     new_fact_ep_ids: dict
7     log: ConsolidationLogEntry    # carried as a dict during execution

```

Listing 4.4: ConsolidationState

4.3.2 Graph Wiring

```

1 g = StateGraph(ConsolidationState)
2 g.add_node("extract", _extract)
3 g.add_node("dedupe", _dedupe)
4 g.add_node("contradiction_check", _contradiction_check)
5 g.add_node("prune", _prune)
6 g.set_entry_point("extract")
7 g.add_edge("extract", "dedupe")
8 g.add_edge("dedupe", "contradiction_check")
9 g.add_edge("contradiction_check", "prune")
10 g.add_edge("prune", END)

```

Listing 4.5: Linear consolidation graph

There are no conditional edges — every run passes through all four stages. The compiled graph is memoized once at module load (`_graph`), not recompiled per request.



4.3.3 Node 1 — `_extract`

For each episode, an LLM call is issued with a strict extraction system prompt: emitted facts must be full, standalone sentences (not fragments), each tagged with one of the six `FactType` values, and the reply must be a bare JSON array (no prose). The reply is parsed with `json.loads`; any per-episode parse failure is silently swallowed (`except Exception: pass`) so one malformed LLM response cannot abort the whole batch.

4.3.4 Node 2 — `_dedupe`

Each extracted candidate fact is embedded and compared by cosine similarity against every existing non-superseded fact; the best match is tracked.

- If $\text{best_sim} \geq \text{SIM_DEDUPE} = 0.92$ (hardcoded — deliberately tighter than the 0.60 contradiction-detection threshold, since deduplication requires near-identical restatement rather than mere topical relatedness), the candidate is treated as a *re-observation* of the matched fact:

$$\text{confidence}_{\text{new}} = \text{round}(w \cdot 1.0 + (1 - w) \cdot \text{confidence}_{\text{old}}, 4)$$

where $w = \text{memory.ema_new_weight}$ (default 0.7). This is an exponential moving average that treats each re-observation as a fresh `confidence = 1.0` sample, pulling the running confidence toward 1.0 with weight w while retaining $(1 - w)$ of the prior value. `last_seen` is refreshed and a new provenance link is added; `log.facts_updated` increments.

- Otherwise a brand-new `Fact` is created (confidence 1.0, `last_seen` set to the *source episode's own timestamp*, not wall-clock now) and its provenance episode ID is stashed for the next stage; `log.facts_created` increments. New facts are **not yet persisted** at this point — persistence is deferred to the contradiction-check stage, since a same-run collision between two new facts must still be resolvable.

4.3.5 Node 3 — `_contradiction_check`

For each new candidate fact, `contradiction.find_contradictions()` is run against a *live, continuously updated* view of `existing_facts`:

- **Conflict found:** `contradiction.resolve()` is invoked; a `contradiction_resolved` detail row is logged (winner/loser IDs and content); the new episode's provenance link is attached to the *winner*; if the winner is the brand-new fact (not previously indexed), it is embedded and upserted into Qdrant now.
- **No conflict:** the new fact is persisted directly to SQLite, its provenance saved, its embedding upserted into Qdrant, and it is appended to the local `existing_facts` list.

Updating `existing_facts` in place after every resolution is deliberate: it allows two *new* facts extracted within the same consolidation run to contradict each other, not just a new fact against pre-existing memory.

4.3.6 Node 4 — `_prune`

All active (non-superseded) facts are scanned; any with `confidence < memory.confidence_floor` (default 0.1) are soft-deleted — `superseded_by` is set to the literal string "pruned" and the corresponding point is removed from the vector index — and a `pruned` detail row is logged. The SQLite row itself is never removed.

4.3.7 Entry Point

```
1 def run_consolidation(user_id, episodes) -> ConsolidationLogEntry:
2     state = {"user_id": user_id, "episodes": episodes, "extracted": [],
3             "new_facts": [], "new_fact_ep_ids": {}, "log": {...}}
4     final_state = _graph.invoke(state)
5     log_entry = ConsolidationLogEntry(**final_state["log"])
6     sqlite_store.save_consolidation_log(log_entry)
7     return log_entry
```

Listing 4.6: `run_consolidation()`

4.4 Why Provenance Never Breaks

The unifying design principle across retrieval, contradiction, and pruning is that **no fact row is ever physically deleted**. Superseding (via contradiction) and pruning (via confidence decay) both operate by setting `superseded_by` and removing only the *vector index entry* — the fact remains queryable in SQLite, and every `fact_provenance` link that pointed at it remains valid. This means `get_provenance(fact_id)` and the MCP `get_provenance` tool can always answer “why does the assistant believe this / why did it stop believing this” for the full history of the memory store, not just its current live state.

Chapter 5

LLM Client Abstraction

5.1 Uniform Interface (`src/llm_client.py`, 160 lines)

Every call site in the codebase (chat, extraction, contradiction judging, roadmap synthesis) talks to one of two functions:

```
1 def chat(messages: list[dict], system: str | None = None,
2         temperature: float = 0.2) -> str: ...
3
4 def stream(messages: list[dict], system: str | None = None,
5            temperature: float = 0.2) -> Iterator[str]: ...
```

Listing 5.1: Public interface

The active provider is read fresh on every call from `config.get("llm.provider", "groq")` and dispatches to a provider-specific implementation. An unrecognized provider name raises `ValueError`.

5.2 Provider Implementations

Provider	Implementation notes
Groq	groq.Groq SDK client; model from <code>llm.groq_model</code> (default <code>llama-3.1-8b-instant</code>). Streaming iterates response chunks, pulling <code>chunk.choices[0].delta.content</code> .
Gemini	<code>google.generativeai</code> ; builds <code>GenerativeModel(model_name, system_instruction=system)</code> , converts chat history into <code>{"role": "user" "model", "parts": [...]}</code> turns, uses <code>start_chat(history=...)</code> then <code>send_message()</code> . Streaming uses <code>generate_content(contents, stream=True)</code> .
Ollama	Raw <code>httpx.post</code> to <code>{ollama_base_url}/api/chat</code> (default <code>http://localhost:11434</code>) with <code>stream: false/true</code> ; parses <code>.json()["message"]["content"]</code> , or NDJSON lines when streaming. Requires no API key — fully local.

All three SDKs are imported *lazily inside the function body* rather than at module top level, so a deployment using only Groq, for instance, never needs the Gemini or Ollama client libraries installed.

Chapter 6

HTTP API ([src/api.py](#))

FastAPI application, title “*mnemos API v0.1.0*”. On startup (lifespan), `sqlite_store.init_db()` runs, and if `seed_demo: true` in [config.yaml](#), `_seed_demo()` populates five canned episodes about learning Python and immediately runs consolidation, giving a populated first-run demo. CORS is opened for `localhost:5173` (Vite dev) and `localhost:3000` (Docker frontend).

6.1 Request/Response Models

Defined inline in [api.py](#) (not in [schema.py](#)):

```
1 class ChatRequest(BaseModel):
2     message: str
3     session_id: Optional[str] = None
4
5 class ChatResponse(BaseModel):
6     reply: str
7     session_id: str
8     episode_id: str
9     memory_used: list[str]
10
11 class ConsolidateRequest(BaseModel):
12     since_hours: Optional[int] = None
13
14 class PlanRequest(BaseModel):
15     topic: str
16     background: str
17     session_id: Optional[str] = None
18
19 class IngestRequest(BaseModel):
20     source: str
21     kind: str = "file"
22     session_id: Optional[str] = None
```

6.2 Endpoint Reference

Method	Path	Description
GET	/health	Liveness probe, returns {"status": "ok"}.

Method	Path	Description
POST	/chat	Retrieves the top-5 decay-ranked facts via <code>retrieval.retrieve_for_context</code> , reconstructs the last 10 episodes of session history, calls <code>llm_client.chat()</code> with a system prompt injecting the retrieved memory context, persists the turn as a new <code>Episode</code> , and invokes <code>_maybe_auto_consolidate()</code> .
POST	/chat/stream	Same retrieval/history construction as /chat, but streams tokens over Server-Sent Events as <code>data: {"token": ...}</code> , followed by a final <code>data: {"done": true, episode_id, session_id, memory_used}</code> . Auto-consolidation, if triggered, runs in a background <code>threading.Thread</code> <i>after</i> the done event so it never holds the SSE connection open.
GET	/memory/facts	Query param <code>include_superseded: bool = False</code> . Lists facts.
GET	/memory/fact/{fact_id}	Returns the fact plus its full list of source episodes (all episodes are loaded once and joined in-memory to avoid N+1 queries).
POST	/memory/consolidate	Runs the sleep cycle over episodes since <code>since_hours</code> , or the last 500 unconsolidated episodes if omitted. Returns the resulting <code>ConsolidationLogEntry</code> .
GET	/memory/log	Consolidation run history.
POST	/memory/ingest	Reads a local file (plain text, or PDF via <code>pypdf</code> , truncated to 8000 characters) or fetches a GitHub repository’s metadata + README via the GitHub API, and stores the content as an unconsolidated <code>Episode</code> .
GET	/episodes	Query param <code>limit: int = 50</code> . Raw episodic memory listing.
GET	/goals	Lists all <code>GoalFact</code> roadmap phases.
PATCH	/goals/{goal_id}/status	Query param <code>status: str</code> ; validates membership in <code>{not_started, in_progress, done}</code> , returning 404/400 on an unknown ID or invalid status.
POST	/plan	Stores the planning request as an <code>Episode</code> , invokes <code>plan_roadmap()</code> (the researcher+synthesizer <code>LangGraph</code>), and returns the generated phases as <code>GoalFacts</code> .

6.3 Auto-Consolidation Trigger

```

1 def _maybe_auto_consolidate():
2     trigger = get("consolidation.trigger", "manual")    # manual /
3     end_of_session / both
4     if trigger == "manual":
5         return
6     last_run = sqlite_store.get_consolidation_log(_USER_ID, limit=1)
7     since = last_run[0].timestamp if last_run else None

```

```

7     new_episodes = sqlite_store.get_episodes_since(_USER_ID, since) if since
8         \
9         else sqlite_store.get_episodes(_USER_ID, limit=10_000)
10    if len(new_episodes) >= get("consolidation.min_episodes_to_trigger", 3):
        run_consolidation(_USER_ID, new_episodes)

```

Listing 6.1: `__maybe_auto_consolidate()` logic

6.4 Ingestion Duplication

Note that file/GitHub ingestion logic is implemented *twice* — once inline within the `/memory/ingest` REST endpoint, and once as the independent MCP tools `ingest_file/ingest_github` (§7). Both use the same 8000-character truncation and the same `_TEXT_SUFFIXES` allow-list, but are not factored into a shared module — a candidate for future consolidation of the codebase itself.

Chapter 7

MCP Server

7.1 Purpose and Transport

`src/mcp_server.py` (417 lines) exposes the same underlying memory operations as first-class tools for MCP-compatible clients (Claude Desktop, Cursor), independent of the REST API. Built with `FastMCP(name="mnemos", instructions=...)`. Two transport modes:

```
1 uv run python -m src.mcp_server # stdio (Claude Desktop)
2 uv run python -m src.mcp_server --http --port 8001 # HTTP/SSE
```

Listing 7.1: Running the MCP server

7.2 Tools

Tool	Description
<code>remember(text, session_id="mcp_session")</code>	Stores an Episode; returns a confirmation string containing the new <code>episode_id</code> .
<code>recall(query, top_k=5)</code>	Calls <code>retrieval.retrieve()</code> ; formats a ranked list including type, confidence, decay-adjusted score, last-seen, and a flagged marker.
<code>get_provenance(fact_id)</code>	Traces a fact back to its source episodes; formats timestamps, session IDs, and truncated (300-char) episode text.
<code>consolidate()</code>	Runs the sleep cycle over up to 500 episodes; returns a formatted summary of the resulting <code>ConsolidationLogEntry</code> .
<code>list_facts(include_flagged=True)</code>	Lists all non-superseded facts sorted by confidence descending, with an optional flagged-only filter.
<code>plan_learning_roadmap(topic, background)</code>	Invokes <code>plan_roadmap()</code> ; formats phases with resources, status, and goal ID.
<code>ingest_file(path, session_id="file_ingest")</code>	Reads a local text/PDF file (same 8000-char truncation and suffix allow-list as the REST endpoint) and stores it as an Episode.

Tool	Description
<code>ingest_github(repo_url, session_id="github_ingest")</code>	Regex-parses owner/repo from the URL, fetches GitHub API metadata + README (optional GITHUB_TOKEN bearer auth), stores as an Episode.

7.3 Resources

Resource URI	Content
<code>memory://facts</code>	Markdown table of all active facts: Type Confidence Last Seen Content .
<code>memory://consolidation-log</code>	Plain-text summary of the 10 most recent consolidation runs, one line per run.

Chapter 8

Roadmap Planner

8.1 Purpose

Given a topic and the user's background, [src/planner/roadmap_planner.py](#) (240 lines) produces a multi-phase learning roadmap grounded in live web/arXiv/GitHub research rather than pure LLM recall, via a two-node LangGraph.

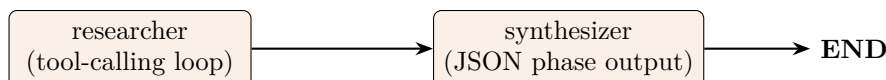
8.2 State Schema

```
1 class PlannerState(TypedDict):
2     messages: Annotated[list, operator.add]    # LangGraph auto-accumulates
3     topic: str
4     background: str
5     raw_plan: str    # JSON string produced by the synthesizer
```

Listing 8.1: PlannerState

8.3 Graph Wiring

```
1 g = StateGraph(PlannerState)
2 g.add_node("researcher", _researcher)
3 g.add_node("synthesizer", _synthesizer)
4 g.set_entry_point("researcher")
5 g.add_edge("researcher", "synthesizer")
6 g.add_edge("synthesizer", END)
```



8.4 Node 1 — Researcher

Builds a tool-bound chat model via `_get_llm_with_tools()` — either `ChatGroq` or `ChatGoogleGenerativeAI` with `.bind_tools(PLANNER_TOOLS)`. Ollama is explicitly unsupported here (`ValueError`) since the local integration used does not support tool calling. Runs an agentic loop capped at 8 iterations:

```
1 for _ in range(8):
2     response = llm.invoke(messages)
3     messages.append(response)
```



```

4     if not response.tool_calls:
5         break
6     tool_results = ToolNode(PLANNER_TOOLS).invoke(messages)
7     messages.extend(tool_results)
8     return {"messages": messages[2:]}    # drop initial system/human turns

```

Listing 8.2: Agentic tool-calling loop (simplified)

The system prompt instructs 5–8 total tool calls: `web_search` for learning resources, `github_search` for reference repositories, `arxiv_search` at least twice with distinct CS category codes, and an optional `fetch_page` for deeper reading. Returning `messages[2:]` avoids LangGraph’s `operator.add` reducer duplicating the initial system/human turns in the accumulated state.

8.5 Node 2 — Synthesizer

Builds a *fresh*, non-tool-bound LLM at `temperature=0.0` (deterministic structured output), concatenates all AI/tool message content from the research history (capped to the last 6000 characters), and prompts for a strict JSON array of 4–6 phases:

```

1  [
2    {
3      "phase_index": 0,
4      "phase_content": "...",
5      "resources": ["https://...", "..."],
6      "prerequisites": "...",
7    }
8  ]

```

Listing 8.3: Synthesizer output shape

The prompt explicitly forbids repeating the same resource URL across phases.

8.6 Post-Processing (`plan_roadmap`)

```

1  def plan_roadmap(topic: str, background: str, user_id: str | None = None,
2                  source_episode_id: str | None = None) -> list[GoalFact]:
3      ...

```

Listing 8.4: `plan_roadmap()` signature

After invoking the graph: markdown code fences are stripped from `raw_plan`; `json.loads` is attempted, falling back to a regex `\[.*\]` extraction if parsing fails; a **cross-phase URL dedup pass** runs in Python, normalizing arXiv URLs by stripping trailing version suffixes (`rstrip("v0123456789")`) so `.../2301.01234v1` and `.../2301.01234v2` are recognized as the same resource; each phase becomes a `GoalFact` (`status="not_started"`) persisted via `sqlite_store.save_goal_fact`, optionally linked to `source_episode_id` for provenance.

8.7 Research Tools (`src/planner/tools.py`, 182 lines)

LangChain `@tool`-decorated functions bound to the researcher LLM as `PLANNER_TOOLS`.

Tool	Behavior
<code>web_search(query)</code>	Tavily API (<code>TAVILY_API_KEY</code> , max 5 results); silently falls back to <code>duckduckgo_search.DDGS().text()</code> on any exception or missing key.

Tool	Behavior
github_ search(query)	GitHub REST search/repositories?sort=stars&order=desc&per_page=5, optional GITHUB_TOKEN bearer auth; formats name/stars/description/topics/URL.
fetch_page(url)	httpx.get with a custom User-Agent; BeautifulSoup strips script/style/nav/footer/header, collapses whitespace, truncates to 3000 characters via textwrap.shorten.
arxiv_search(query, categories="cs.LGcs. AICS.CLstat.ML")	Builds a scoped query (query) AND (cat:X OR cat:Y ...); uses arxiv.Client() / arxiv.Search(sort_by=Relevance,max_results=10); post-filters to a hardcoded category allow-list (cs.LG,cs.AI,cs.CL,cs.CV,cs.NE,cs.IR,cs.HC,cs.RO,cs.SE,cs.CR,stat.ML,econ.EM,q-bio.NC); returns up to 4 results (title, categories, authors, published date, cleaned URL, abstract). Falls back to an unfiltered search if category filtering yields zero results.

Chapter 9

Configuration

9.1 Loader ([src/config.py](#), 41 lines)

```
1 @lru_cache(maxsize=1)
2 def load_config() -> dict:
3     ... # loads config.yaml once, resolves database/vector_store paths
4         # to absolute paths against the repo root
5
6 def get(key_path: str, default=None):
7     ... # dotted-path lookup, e.g. get("memory.ema_new_weight")
8
9 def api_key(provider: str) -> str | None:
10    # groq -> GROQ_API_KEY, gemini -> GOOGLE_API_KEY env vars
11    # ollama needs no key
```

Listing 9.1: Config loading

`.env` is loaded via `python-dotenv` at import time. Path resolution anchors on `_ROOT = Path(_file_).parent.parent` so relative paths in [config.yaml](#) always resolve consistently regardless of the process's working directory.

9.2 Full Tunable Reference

```
1 llm:
2   provider: groq                # groq | gemini | ollama
3   groq_model: llama-3.1-8b-instant
4   gemini_model: gemini-1.5-flash
5   ollama_model: llama3
6   ollama_base_url: http://localhost:11434
7 embeddings:
8   model: all-MiniLM-L6-v2       # sentence-transformers, CPU
9   dimension: 384
10 vector_store:
11   path: ./data/qdrant
12   collection: mnemos_facts
13 database:
14   path: ./data/mnemos.db
15 memory:
16   ema_new_weight: 0.7           # EMA weight toward a fresh observation
17   decay_half_life_days: 30      # retrieval recency half-life
18   confidence_floor: 0.1         # prune threshold
19   similarity_threshold: 0.60    # contradiction pre-filter (cosine)
20 consolidation:
21   trigger: manual               # manual | end_of_session | both
22   min_episodes_to_trigger: 3
```

```
23 user:
24   default_user_id: local_user
25 seed_demo: false
```

Listing 9.2: config.yaml

Documented inconsistency. The frontend footer status bar ([App.jsx](#)) displays static, hardcoded pipeline parameters ($t_{1/2}$ 168h, $\text{sim} \geq 0.85$, floor 0.15) that do not match the values actually in force per [config.yaml](#) (30-day half-life, 0.60 similarity threshold, 0.10 confidence floor). These display values are not live-fetched from `/health` or any config endpoint — a UI/documentation gap rather than a functional bug, since the backend always uses the real [config.yaml](#) values regardless of what the footer displays.

Chapter 10

Frontend

React 19 + Vite, JSX only (no TypeScript), no CSS framework — all styling is hand-written CSS using HSL custom properties.

10.1 Application Shell ([App.jsx](#), 59 lines)

Four tabs — chat, memory, roadmap, log — rendered respectively as Chat, MemoryInspector, Goals, ConsolidationLog. A single `SESSION_ID = crypto.randomUUID()` is generated once at module scope per page load (not per render), scoping all chat activity for that browser tab to one session.

10.2 API Layer ([api.js](#), 65 lines)

A thin axios wrapper (`baseUrl: ""`), relying on the Vite dev-server proxy (or, in Docker, nginx) to forward `/chat`, `/memory`, `/episodes`, `/goals`, `/plan`, `/health` to the backend on the same origin.

```
1 export async function streamMessage(message, sessionId, onToken, onDone) {
2   const res = await fetch('/chat/stream', {
3     method: 'POST',
4     headers: { 'Content-Type': 'application/json' },
5     body: JSON.stringify({ message, session_id: sessionId }),
6   });
7   const reader = res.body.getReader();
8   const decoder = new TextDecoder();
9   let buffer = '';
10  while (true) {
11    const { done, value } = await reader.read();
12    if (done) break;
13    buffer += decoder.decode(value, { stream: true });
14    const lines = buffer.split('\n');
15    buffer = lines.pop(); // retain partial trailing line
16    for (const line of lines) {
17      if (!line.startsWith('data: ')) continue;
18      const payload = JSON.parse(line.slice(6));
19      if (payload.done) onDone(payload);
20      else onToken(payload.token);
21    }
22  }
23 }
```

Listing 10.1: Hand-rolled SSE stream reader

Other exported functions: `sendMessage`, `getFacts`, `getFactWithProvenance`, `triggerConsolidate`, `getConsolidationLog`, `getEpisodes`, `getGoals`, `updateGoalStatus`, `createRoadmap`, `ingestSource`.

10.3 Panels

10.3.1 Chat ([panels/Chat.jsx](#), 301 lines)

Streaming message list where assistant replies grow token-by-token via the `streamMessage` callback appending to the last message's content. Each assistant message has an expandable “memory” panel showing the `fact_id/content/score` that informed that specific reply. A collapsible “ctx” strip at the top shows the top-8 facts loaded on mount (`getFacts()`). Supports copy-to-clipboard per message, an auto-resizing textarea, Enter-to-send, and a roadmap-generation modal (topic + background → `createRoadmap`). `confBar(conf)` maps a confidence value to an HSL teal color for the visual confidence bars.

10.3.2 Memory Inspector ([panels/MemoryInspector.jsx](#), 188 lines)

A toolbar with a file/GitHub ingest form (`ingestSource`), type-filter pills (`all/preference/skill/status/event/goal/other`), and a manual “consolidate” trigger. The main view is a fact table (type, content, confidence %, age) whose rows expand on click to reveal confidence, last-seen, source count, the raw `fact_id`, and the full list of provenance episodes (lazily fetched via `getFactWithProvenance`).

10.3.3 Goals ([panels/Goals.jsx](#), 122 lines)

A three-column Kanban board — TODO/ACTIVE/DONE mapped to `not_started/in_progress/done` — with cards grouped by topic and ordered by `phase_index` within each status column. Each card shows the topic, phase description, up to two resource links, and status-transition buttons (`updateGoalStatus`). The top bar reports topic count and a done/total completion fraction.

10.3.4 Consolidation Log ([panels/ConsolidationLog.jsx](#), 135 lines)

A timeline of consolidation runs; each entry expands into a synthetic “terminal log” view (`buildLogLines`) rendering the four pipeline stages (`extract` → `dedupe` → `contradict` → `prune`) with their respective counts, plus up to two contradiction-detail lines showing the winning fact's content. Includes a “run now” button wired to `triggerConsolidate`.

10.4 Design System ([index.css](#), 68 lines)

Fonts: IBM Plex Sans (UI text) and IBM Plex Mono (data/numeric values). The palette is a dark, zero-radius “terminal” aesthetic:

```
1 :root {
2   --bg: hsl(0, 0%, 2%); /* page background, near-black */
3   --s1: hsl(0, 0%, 5%); /* surface 1 */
4   --s2: hsl(0, 0%, 8%); /* surface 2 */
5   --s3: hsl(0, 0%, 12%); /* surface 3 */
6   --b1: hsl(0, 0%, 14%); /* border 1 */
7   --b2: hsl(0, 0%, 24%); /* border 2 */
8   --b3: hsl(0, 0%, 38%); /* border 3 */
9   --t1: hsl(0, 0%, 95%); /* text primary */
10  --t2: hsl(0, 0%, 68%); /* text secondary */
11  --t3: hsl(0, 0%, 46%); /* text tertiary */
12  --t4: hsl(0, 0%, 28%); /* text quaternary / dim */
13  --ac: hsl(180, 100%, 50%); /* accent -- teal / cyan */
14  --am: hsl(24, 100%, 58%); /* amber -- logo underscore, warnings */
15  --gr: hsl(140, 80%, 48%); /* green -- success / ok */
16  --re: hsl(0, 90%, 60%); /* red -- error */
17  --f: 'IBM Plex Sans', system-ui, sans-serif;
```

```
18  --fm:  'IBM Plex Mono', monospace;
19  --tx: 11px; --ts: 12px; --tm: 13px; --tl: 14px; --txl: 15px; /* type scale
    */
20  --r:  0px;  --rl: 0px;      /* border-radius: none, anywhere    */
21 }
```

Listing 10.2: Root design tokens

Confidence visualizations elsewhere reuse the accent hue: `hsl(180, saturation=conf%, lightness=18+conf*32)`, so higher-confidence facts render as more saturated, brighter teal bars.

Chapter 11

Deployment & Tooling

11.1 Docker Compose Topology

```
1 services:
2   backend:
3     build: .
4     env_file: .env
5     volumes: ["/data:/app/data"]
6     ports: ["8000:8000"]
7     healthcheck: curl -f http://localhost:8000/health
8
9   frontend:
10    build: ./frontend
11    ports: ["3000:80"]
12    depends_on:
13      backend:
14        condition: service_healthy
```

Listing 11.1: docker-compose.yml (structure)

11.2 Backend Dockerfile

Base image `python:3.11-slim`; installs `curl` (for the healthcheck) and `uv`. Three cache-friendly layers:

1. `uv sync -frozen -no-install-project` run against just `pyproject.toml/uv.lock`, copied first — so dependency installation is cached across code-only changes.
2. The `all-MiniLM-L6-v2` embedding model is baked into the image at *build time* (`python -c "SentenceTransformer(...)"`), so container cold-starts never pay the model-download cost.
3. `src/` and `config.yaml` are copied last, since they change most often.

Entrypoint: `uvicorn src.api:app -host 0.0.0.0 -port 8000`; healthcheck runs every 15 seconds.

11.3 Frontend Dockerfile

Multi-stage: a `node:20-alpine` builder stage runs `npm ci` then `npm run build` (Vite), followed by an `nginx:alpine` runtime stage serving `/app/dist` with a custom `nginx.conf`; healthcheck via `wget`.

11.4 Local Development (Makefile)

```
1 install: # uv sync + npm install
2 dev: # runs backend and frontend concurrently (bash), trap 'kill 0' EXIT
3 backend: # uv run uvicorn src.api:app --reload -> :8000
4 frontend: # cd frontend && npm run dev -> :5173
```

Listing 11.2: Makefile targets

11.5 Python Dependencies (pyproject.toml)

Package `mnemos 0.1.0`, requires Python ≥ 3.11 , build backend `hatchling`.

Category	Packages
Web framework	<code>fastapi</code> , <code>uvicorn[standard]</code> , <code>pydantic>=2.7</code>
Config/env	<code>python-dotenv</code> , <code>pyyaml</code>
LLM providers	<code>groq</code> , <code>google-generativeai</code>
Embeddings/vectors	<code>sentence-transformers</code> , <code>qdrant-client</code> , <code>numpy</code>
Orchestration	<code>langgraph</code> , <code>langchain-core</code> , <code>langchain-groq</code> , <code>langchain-google-genai</code> , <code>langgraph-prebuilt</code>
Research tools	<code>httpx</code> , <code>duckduckgo-search</code> , <code>arxiv</code> , <code>beautifulsoup4</code>
MCP & docs	<code>fastmcp</code> , <code>pypdf</code>
Dev tooling	<code>pytest</code> , <code>pytest-asyncio</code> , <code>ruff</code> (dev extras)

Console script: `mnemos-mcp = src.mcp_server:mcp.run`. `Ruff` is configured with `line-length = 100`, `target-version = "py311"`.

11.6 Environment Variables

Variable	Required	Notes
<code>GROQ_API_KEY</code>	At least one LLM key	Free tier available
<code>GOOGLE_API_KEY</code>	At least one LLM key	Gemini free tier
<code>TAVILY_API_KEY</code>	Optional	Web search; DuckDuckGo fallback if absent
<code>GITHUB_TOKEN</code>	Optional	Raises GitHub API rate limit from 60 to 5000 req/hr
<code>LANGSMITH_API_KEY</code>	Optional	LangSmith tracing
<code>LANGCHAIN_TRACING_V2</code>	Optional	Enables LangSmith tracing
<code>LANGCHAIN_PROJECT</code>	Optional	LangSmith project name

Ollama requires no API key since it runs entirely locally.

Chapter 12

Cross-Cutting Observations

1. **Single-tenant by design.** `user_id` is always the same config-driven constant throughout `api.py` and `mcp_server.py`; there is no authentication or multi-tenancy layer anywhere in the system. This is appropriate for a personal, self-hosted deployment but would need to be introduced before any multi-user use case.
2. **Duplicated ingestion logic.** File and GitHub ingestion are implemented independently in the REST `/memory/ingest` endpoint and in the MCP `ingest_file/ingest_github` tools, sharing constants (the 8000-char truncation limit, the text-suffix allow-list) but not a common code path.
3. **Provenance is permanent.** Both pruning and contradiction resolution use soft-supersession (`superseded_by`) rather than deletion — see §4.4. This is the central architectural commitment of the memory subsystem.
4. **Both LangGraph graphs are compiled once.** The consolidation graph (§4.3) and the roadmap planner graph (Chapter 7) are each memoized at module load into a module-level global, not recompiled per request.
5. **Config/UI drift.** The frontend footer’s displayed pipeline parameters do not match `config.yaml`’s actual values (§6.2) — a cosmetic inconsistency, not a functional one, since the backend always reads the real config.
6. **Threshold asymmetry is intentional.** The 0.92 hardcoded deduplication threshold in `_dedupe` is deliberately tighter than the 0.60 configured contradiction-detection threshold: deduplication requires near-verbatim restatement, while contradiction detection needs only topical relatedness before the LLM judge is asked to confirm genuine conflict.